



instead of code add mermaid flowcharts abd algorithms in english

Here is a concise guide on each of the 12 DSAL practicals for SPPU, replacing code examples with mermaid flowcharts (for concepts/structures) and stepwise algorithms in simple English. This will help with oral and viva-focus on process understanding!

1. Telephone Book Database (Hash Table)

Algorithm Steps

1. Choose a hash function for mapping names to indices.
2. On insertion:
 - Calculate index via hash.
 - If slot is empty, insert.
 - If full, handle collision (linear probing or chaining).
3. On search, compute index and check if data matches (probe/chain if not found).
4. On deletion, locate and mark entry as removed.

Flowchart (Mermaid)

```
graph TD
    A[Start] --> B[Input name & phone]
    B --> C[Hash function -> index]
    C --> D{Slot empty?}
    D -- Yes --> E[Insert record]
    D -- No --> F[Handle collision]
    F --> E
    E --> G[End]
```

2. Dictionary ADT (Hashing & Chaining)

Algorithm Steps

1. Hash the key to compute the index.
2. Traverse linked list at index for insertion/search/deletion.
3. For insertion: append new node to end of chain.
4. For search: scan chain nodes for key.
5. For delete: remove node from chain.

Flowchart (Mermaid)

```
graph TD
    A[Input key, value] --> B[Hash -> index]
    B --> C{Chain exists?}
    C -- Yes --> D[Traverse list]
    C -- No --> E[Insert new node]
    D --> F{Key found?}
    F -- Yes --> G[Update/Delete]
    F -- No --> E
    E --> H[End]
```

3. Book Structure as Tree

Algorithm Steps

1. Root node is Book; children are Chapters.
2. Each Chapter contains Sections as children.
3. Continue adding nodes recursively for subsections.
4. For printing/display: traverse tree in pre-order.

Flowchart (Mermaid)

```
graph TD
    A[Book] --> B[Chapter 1]
    A --> C[Chapter 2]
    B --> D[Section 1.1]
    B --> E[Section 1.2]
    D --> F[Subsection 1.1.1]
```

4. Binary Search Tree (BST) Construction

Algorithm Steps

1. Start with an empty tree.
2. For each value:
 - If tree is empty, insert as root.
 - Else, compare with root; if smaller, go left; if greater, go right.
 - Recursively insert in correct subtree.

Flowchart (Mermaid)

```
graph TD
    A[Start] --> B[Input value]
    B --> C{Tree is empty?}
    C -- Yes --> D[Insert as root]
    C -- No --> E[Compare with Node]
    E --> F{Less?}
    F -- Yes --> G[Go Left]
    F -- No --> H[Go Right]
    G --> B
    H --> B
```

5. Expression Tree from Prefix

Algorithm Steps

1. Read prefix from right to left.
2. If symbol is operand, push node to stack.
3. If symbol is operator, pop two nodes; make them children of new operator node; push back to stack.
4. Repeat till end; top of stack is root.

Flowchart (Mermaid)

```
graph TD
    A[Read symbol] --> B{Operand?}
    B -- Yes --> C[Push node to stack]
    B -- No --> D[Pop two, attach as children]
    D --> C
    C --> E{More symbols?}
    E -- Yes --> A
    E -- No --> F[Top of stack = Root]
```

6. Threaded Binary Tree

Algorithm Steps

1. For each new node:
 - Insert like BST, but use left/right "threads" for null pointers.
2. During inorder traversal:
 - If right thread, follow to next node instead of stack/recursion.

Flowchart (Mermaid)

```
graph TD
    A[Insert Value] --> B{BST Insert?}
    B -- Yes --> C[Null found?]
    C -- Yes --> D[Create thread to predecessor/successor]
    D --> E[Next insertion]
    C -- No --> F[Move left/right]
    F --> B
```

7. Graph Representation with DFS & BFS

Algorithm Steps

1. For DFS: Visit node, mark as visited, recursively visit all unvisited neighbors.
2. For BFS: Visit node, enqueue neighbors, repeat until all reachable nodes visited.

Flowchart (Mermaid)

```
graph TD
    A[Start at Node] --> B[Mark Visited]
    B --> C{DFS or BFS?}
    C -- DFS --> D[Recursively visit unvisited neighbors]
    C -- BFS --> E[Enqueue neighbors]
    E --> F[Dequeue node, repeat]
    D --> G{All visited?}
    F --> G
    G -- No --> A
    G -- Yes --> H[End]
```

8. Flight Paths Between Cities (Graph + Weights)

Algorithm Steps

1. Each city is a vertex; flight is an edge with weight (time/fuel).
2. Build adjacency list or matrix with weight.
3. Use traversal/search or Dijkstra for shortest path as needed.

Flowchart (Mermaid)

```
graph TD
    A[Cities as vertices] --> B[Read path, assign weight]
    B --> C[Update edge in graph]
    C --> D{More paths?}
    D -- Yes --> B
    D -- No --> E[Traversal/Search as needed]
```

9. Dictionary (AVL/Height Balanced Tree)

Algorithm Steps

1. Insert key-value as in BST.
2. After every insertion/deletion, update height and check balance.
3. If balance factor violated, rotate nodes accordingly (LL, RR, LR, RL).

Flowchart (Mermaid)

```
graph TD
    A[Insert/Delete] --> B[Update heights]
    B --> C[Check balance factor]
    C -- Imbalanced --> D[Rotate tree]
    C -- Balanced --> E[Continue]
    D --> E
```

10. Heap for Max/Min Marks

Algorithm Steps

1. Insert mark, percolate up to maintain heap property (Max/Min).
2. For max/min, return root.
3. For extraction, swap with last, heapify down.

Flowchart (Mermaid)

```
flowchart TD
    A[Insert Value] --> B[Percolate up]
    B --> C[Heap Property Maintained]
    C --> D[Root = Max/Min]
    D --> E[Extract: Swap with last, Percolate down]
```

11. Sequential File – Student Info

Algorithm Steps

1. To add: append at end.
2. To search/delete: sequentially read from start, search for record, and mark as deleted if needed.
3. To display, read all and print undeleted records.

Flowchart (Mermaid)

```
flowchart TD
    A[Add record] --> B[Append at end]
    B --> C[Search/Delete?]
    C -- Yes --> D[Read from start]
    D --> E{Match?}
    E -- Yes --> F[Process (show/delete)]
    E -- No --> G[Continue]
```

12. Indexed Sequential File – Employee Info

Algorithm Steps

1. Maintain regular data file and index file storing key & position.
2. For search, use index to quickly jump to position.
3. For add/delete, update both data and index files.

Flowchart (Mermaid)

```
flowchart TD
    A[Add new record] --> B[Append to data file]
    B --> C[Update index file]
    C --> D[Search?]
    D -- Yes --> E[Look in index, jump to position]
    E --> F[Process record]
```

This format will help you quickly recall algorithms and describe each process clearly in your practical oral exam. Ask for a downloadable PDF if you want this in one file!

*
**

1. selected_image_5225124071951110541.jpg
2. selected_image_2284812355150860076.jpg